

MODULE 5.1 · Concept 2 of 8

Agent Anatomy: Perception, Decision, Action

The four-component loop that defines every agent — from thermostats to GPT-4 powered systems

9 min

Duration

B2 – Understand

Bloom's

L2 – Intermediate

Level

TH-000001

Prerequisite

AI-AG-FN-TH-000002

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHY & WHAT

The Perception-Reason-Act Loop

Every agent, no matter how complex, follows this pattern

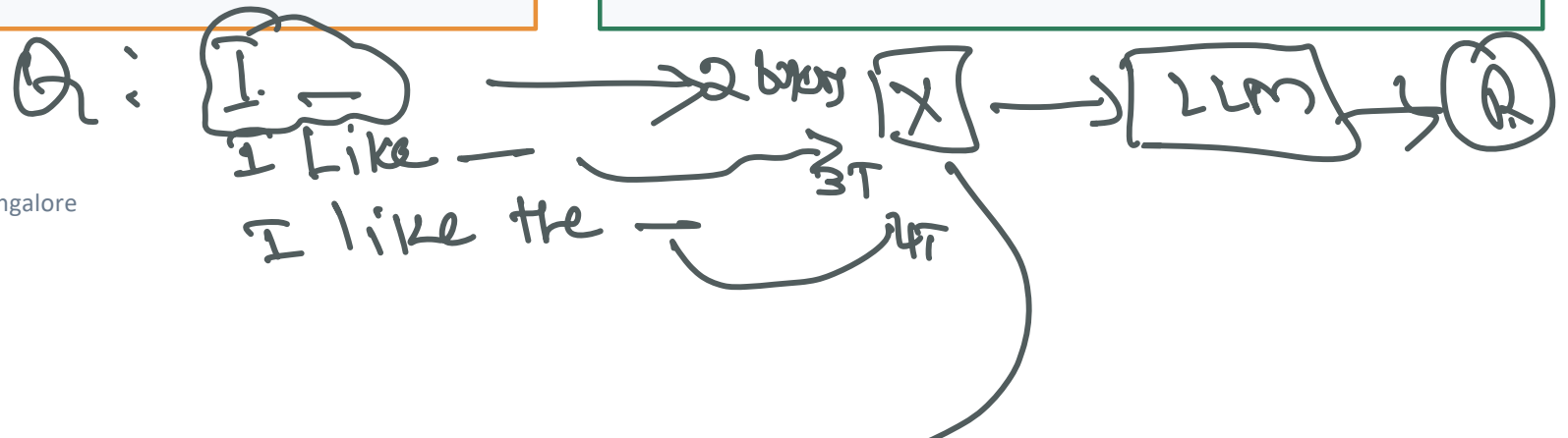
Perception → Memory → Reasoning → Action — the universal agent architecture

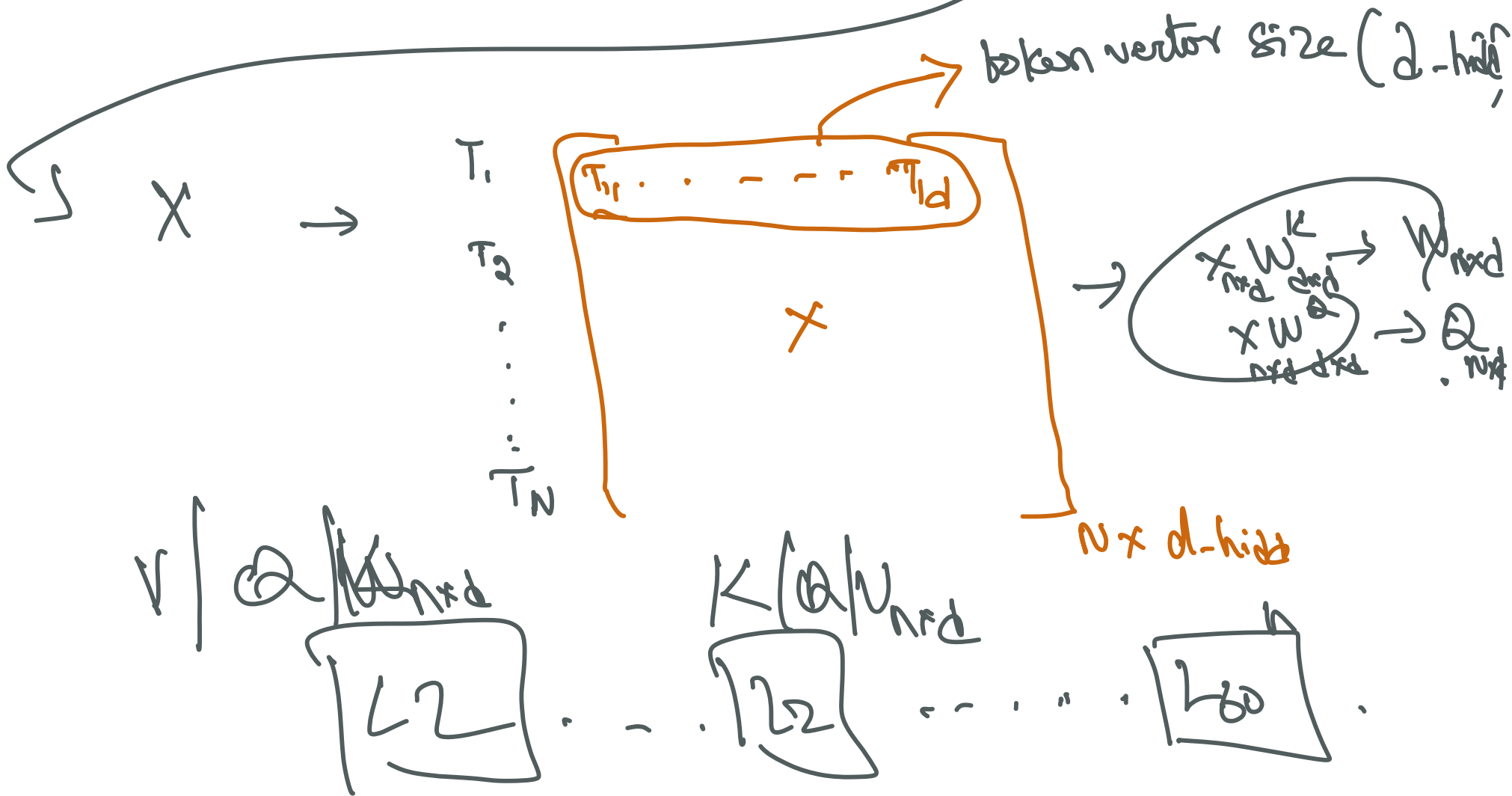
Receives observations: user input, tool outputs, environment state. The agent's eyes and ears. Without perception, the agent is blind to the world.

Short-term (context window, 4K–2M tokens) and long-term (vector DB, Redis). Tracks conversation history and past tasks across sessions.

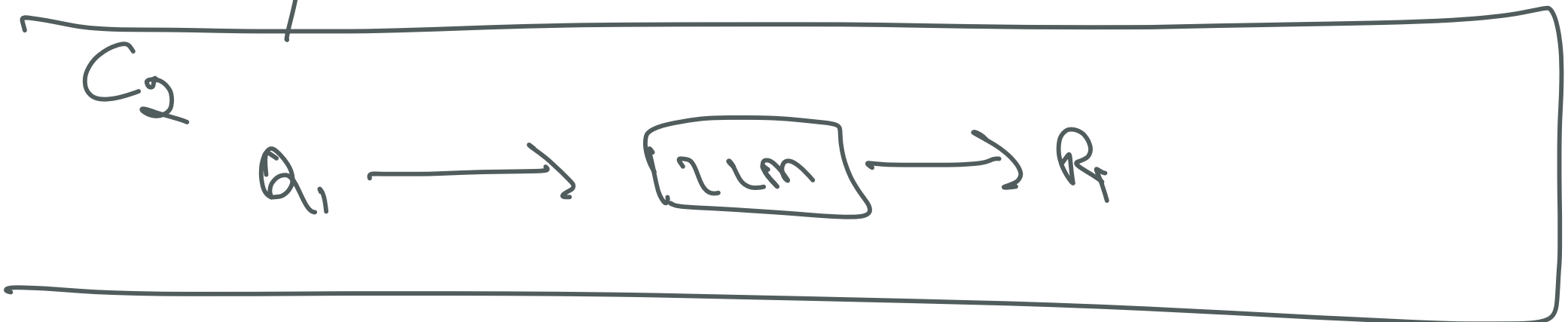
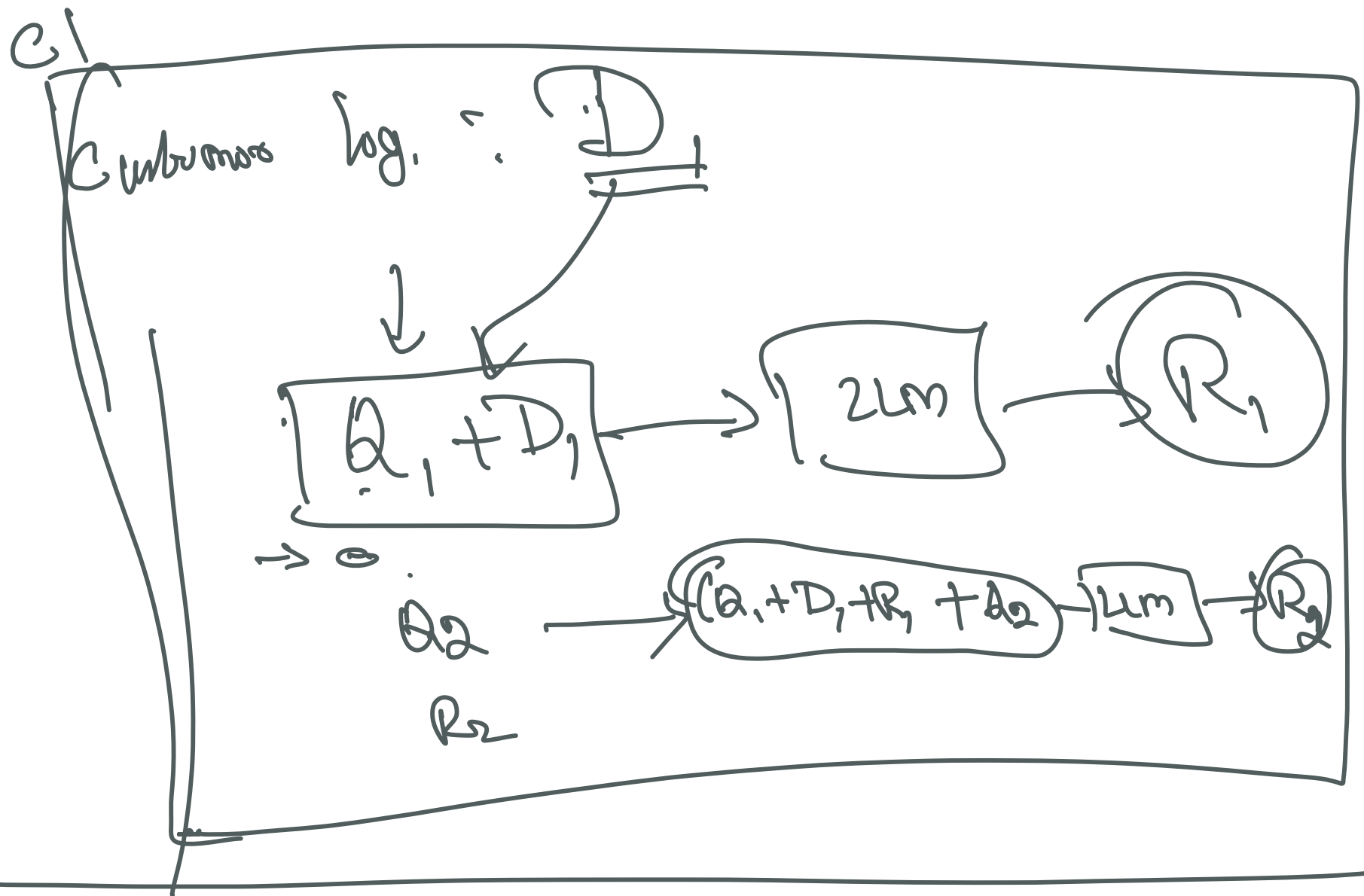
The LLM interprets observations + memory and decides the next action. Uses chain-of-thought to plan. This is the 'brain' of the agent.

Executes tool calls (APIs, code, search), writes files, generates output. Each action produces a new observation → feeds back into the loop.





meet @ 11 am



Reflexive : $T_1 \cdot KB_1, T_1 \cdot KB_2, \dots, T_1 \cdot KB_n$
 $\Rightarrow T_1$ & KB_i are similar

$\{T_1, T_2, T_3\} \quad K_{B_1}, K_{B_{i-1}}, K_{B_i}, K_{B_{i+1}}\}$

$\hookrightarrow Q \rightarrow (Lim) \rightarrow R_1$

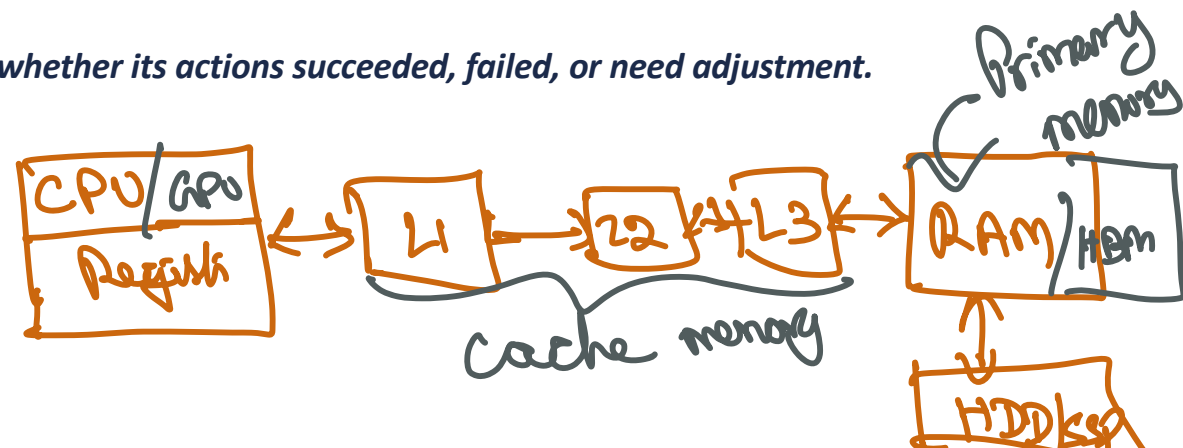
will start @
11.10 am

Deep Dive: Perception

What counts as a percept — and why it's the agent's most critical input

Percept Source	Example	Why It Matters
User Input	Natural language query, uploaded file, voice command	The trigger that starts or continues the agent loop
Tool Output	API response JSON, search results, code execution stdout	Observation from the previous action — feeds back into reasoning
Environment State	File system contents, database state, sensor reading	Context the agent reads to understand current situation
Error Signals	HTTP 500, timeout, invalid parameter, test failure	Critical for self-correction — agent must detect and react to failures
System Events	Timer expiry, webhook trigger, new message in queue	External events that can wake the agent or change its priorities

Without perception, the agent cannot detect whether its actions succeeded, failed, or need adjustment.



Deep Dive: Memory

Short-term context vs. long-term storage — different mechanisms, different tradeoffs

Short-Term (Context Window)

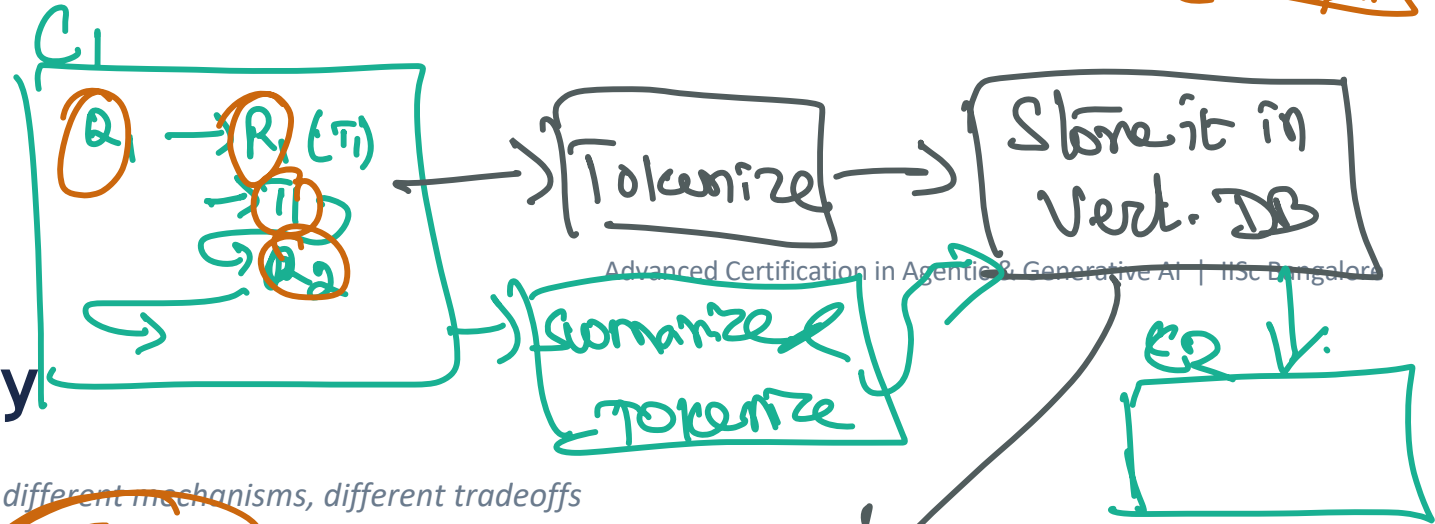
- Holds current conversation + tool outputs
- Bounded: 4K–2M tokens depending on model
- Lost when session ends
- Zero retrieval latency
- Cost scales with token count per call
- Think: RAM — fast but volatile

Long-Term (External Storage)

- Stored in Pinecone, Weaviate, Redis, pgvector
- Retrieved by semantic similarity search
- Persistent — survives restarts & deployments
- Scales to millions of entries
- ~10–50 ms retrieval latency
- Think: Hard disk — slower but permanent

Both types work together: context window for immediate state, external stores for accumulated knowledge.

~ 500 ~ 800 tokens per A4-Page



Deep Dive: Reasoning (The LLM Brain)

How the LLM interprets observations, plans, and decides the next action

The reasoning component does three things every iteration:

- | | | |
|---|------------------|---|
| 1 | Interpret | Read the latest observation + full context. What just happened? Did the last action succeed or fail? Is there new information? |
| 2 | Plan | Given the goal and current state, what should happen next? Chain-of-thought reasoning decomposes the problem into steps. Decides IF another action is needed. |
| 3 | Decide | Select a specific action: which tool to call, with what parameters. Or decide to generate the final answer if the goal is met. |

This is all token prediction — the LLM doesn't 'think'. But conditioned on the right context, it produces remarkably effective decisions.

Deep Dive: Action

How the agent affects the real world — and why each action creates a new observation

Action Type	Example Call	Observation Returned
API Call	search_flights(BLR, BOM, Mar15)	JSON: {flights: [{airline:'IndiGo',...}]}
Code Execution	python run_tests.py	stdout: '3 passed, 1 failed'
Database Query	SELECT * FROM orders WHERE id=4521	{status:'shipped', eta:'tomorrow'}
Web Search	search('Tokyo population 2024')	13.96 million (2024 census)
File Write	write('report.md', content)	File created: 2.3 KB
User Reply	send('Your refund is processed')	Message delivered, user read

Key insight: Every action produces an observation. That observation becomes the next percept. This closes the loop: Action → Observation → Perception → Reasoning → Action.

The Iterative Loop: How Everything Connects

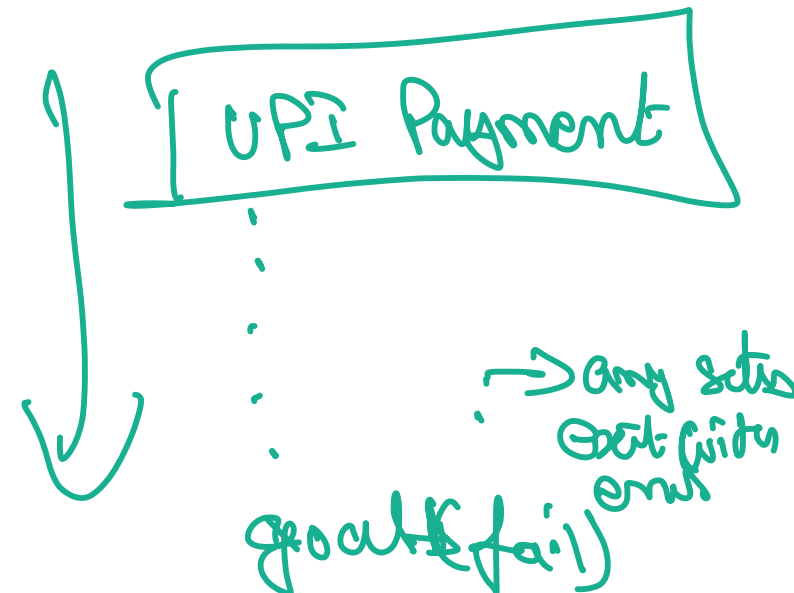
Each action feeds back into perception — the agent never stops until the goal is met



↻ Observation feeds back into Perception ↻

Example: Travel Booking Agent

1. PERCEIVE: User says "Book BLR→BOM under ₹5K" ← SLM
2. REASON: "I need to search flights first" SLM
3. ACT: `search_flights("BLR","BOM","Mar15",max=5000)` Tool
4. OBSERVE: IndiGo ₹4,200 at 6:30 AM ← SLM
5. REASON: "Under budget. I should book this." SLM / classical
6. ACT: `book_flight("IN-4321")` Tool
7. OBSERVE: Confirmation XK9J2 → GOAL MET → Stop ← SLM



SECTION 2 — HOW

Traced Example: Billing Dispute

A customer support agent — 7 steps, 455 tokens, \$0.0003 total cost

Traced Example: Steps 1-4

Customer support agent resolving a duplicate charge

Indicative
↓

Step 1 **Percept** User: "I was charged twice for order #4521"

45 tok

Step 2 **Memory** Retrieve order history from CRM → order total ₹2,499, two charges found

120 tok

Step 3 **Reason** Thought: Duplicate charge confirmed. I should initiate a refund for one charge.

85 tok

Step 4 **Action** call refund_api(order=4521, amount=2499) → Success: refund initiated

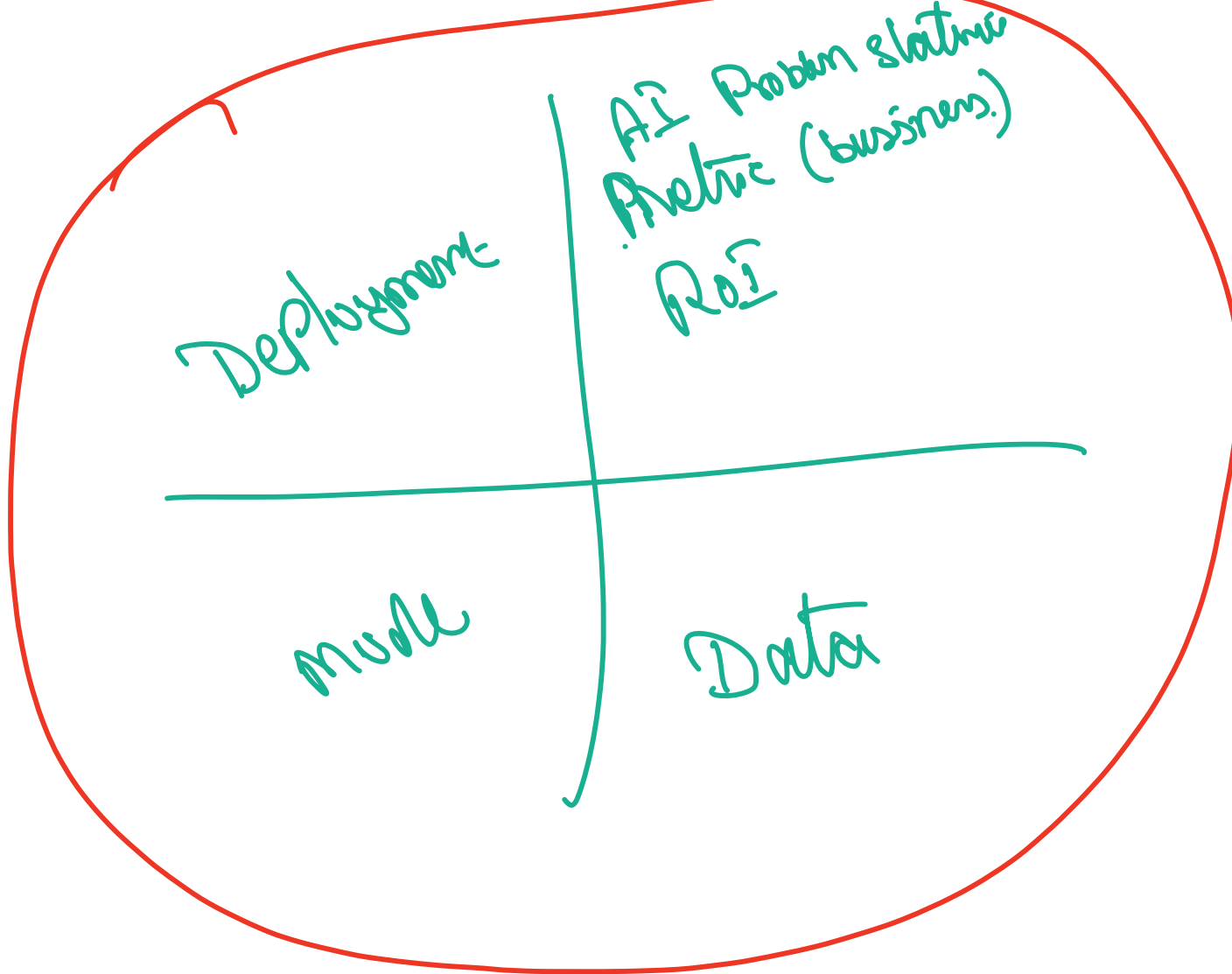
60 tok

Traced Example: Steps 5-7 + Cost Analysis

Completing the resolution and counting the bill

Step 5	Percept	Observation: Refund ID RF-8834 generated, ETA 3–5 business days	40 tok
Step 6	Reason	Thought: Refund done. I should inform the customer with the refund ID.	50 tok
Step 7	Action	Reply: "Refund of ₹2,499 initiated (RF-8834). 3–5 business days."	55 tok

Token Budget: Total = 455 tokens across 7 steps. At GPT-4o-mini pricing (\$0.15/1M in + \$0.60/1M out) $\approx$ **\$0.0003 per resolution.**
At 50K tickets/month = \$15/month for AI resolutions. Compare: Klarna's pre-AI cost = \$8/hr $\times$ 700 agents = \$3.9M/year.



Cost Comparison: Agent vs. Human Resolution

Orders of magnitude difference at scale

\$0.0003

AI agent
per resolution

\$8.00

Human agent
per hour

\$15

AI: 50K tickets
per month

\$325,000

Human: 50K tickets
per month

This is NOT about replacing all humans. Complex cases still escalate. The agent handles routine queries (the 2/3 that follow predictable patterns) at near-zero marginal cost. Humans handle the exceptions.

Mapping Real Systems to the Four Components

Every production agent implements the same anatomy

System	Perception	Memory	Reasoning	Action
GitHub Copilot Workspace	Issue description + codebase AST	File index + recent edits	GPT-4 plans multi-file edit	Writes code + runs tests
Klarna AI Support	Customer message + order data	CRM history + past tickets	Classifies intent, plans resolution	Refunds, escalates, responds
Elicit Research	Research query + paper database	Search results + extracted claims	Synthesizes across multiple papers	Searches, extracts, writes summary

GitHub Copilot Workspace: Component Walkthrough

Tracing the four components through a real coding agent

Perception

Reads the GitHub issue description + scans the entire codebase AST (Abstract Syntax Tree) to understand file structure, dependencies, and relevant code sections.

Memory

Maintains a file index of the repository, tracks which files have been recently modified, stores test results from previous iterations. Context window holds the active editing session.

Reasoning

GPT-4 analyzes the issue, identifies which files need changes, plans a multi-file edit strategy. Decides the order of changes to maintain code consistency.

Action

Writes code changes across multiple files. Runs the test suite. If tests fail, reads error output (new Perception) and reasons about the fix (new Reasoning cycle).

The loop continues: write code → run tests → read errors → reason about fix → write again → until all tests pass.

Key Takeaways

The essential points from this concept

- Every agent has four components:** Perception, Memory, Reasoning (LLM), and Action (tools).
- The loop is iterative:** each action produces an observation that feeds back into perception.
- Real systems map cleanly:** Copilot (code), Klarna (support), Elicit (research) — same anatomy.
- Token budget math:** a full resolution costs $< \$0.001$ — orders of magnitude cheaper than human agents.

Next: AI-AG-FN-TH-000003 — Simple Reactive Agents

AI-AG-FN-TH-000002 Complete

Agent Anatomy: Perception, Decision, Action

Concept 2 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi